Chapter 2

GRAPHICAL MODELS

Graphical models represent geometric information in terms of points and lines. As many of the techniques for processing graphical models are of interest also to solid modeling, it is appropriate to study them more closely.

2.1 BASIC MODELING PRIMITIVES

The construction of a graphical model is based on a selection of *modeling primitives*, and a collection of *modeling procedures* for their instantiation and manipulation. Let us start by examining the very basic modeling primitives of graphical models: points and lines.

2.1.1 Points

The simplest geometric object that we must be able to represent is a location in space, a single point. We shall use a 3-dimensional right-handed Cartesian coordinate system with axes x, y, and z to indicate a location in space, although other coordinate systems (such as cylindrical coordinates) can be of interest in special cases.

To represent and process points in a computer, we shall use the so-called *homogeneous coordinate representation* that adds a fourth coordinate w, as explained in Appendix A. Homogeneous coordinates are convenient because many operations of interest can be performed simply by multiplying the vector $[x\ y\ z\ w]$ with a 4-by-4 matrix. In particular, arbitrary sequences of *translation*, *rotation*, and *scaling* operations on x, y, and z can be combined by multiplying together appropriate 4-by-4 matrices explained in the Appendix.

```
typedef struct
{
        float       vx, vy, vz, vw;
        /* other information can be added here */
} point;

ex1()
{
        point       *p;

        p = newpoint(0.0, 0.0, 0.0);
}
```

Program 2.1 Type *point*.

To represent the point $(x\ y\ z)$, we need a data type *point*, having components vx, vy, vz, and vw. Program 2.1 gives a definition in C of this information. The procedure newpoint is assumed to allocate storage for a new *point* node and store the coordinate values into it. Note that points may be associated with other, problem-dependent data. In Program 2.1, we have represented only the essential geometric data explicitly.

2.1.2 Lines

Points alone would be quite inconvenient for modeling interesting figures. As they have just a location in space, we would not be able to create and process geometric objects having extent. The usefulness of points comes from the observation that straight line segments (that consist of infinitely many points) can be represented indirectly but completely in terms of their end points. A straightforward approach is to represent line segments in terms of two points. In this approach points can be created only as a part of a line segment. Program 2.2 gives the C definition of a type *line* that implements this, using the definitions of Program 2.1.

2.1.3 Independent Points and Lines

When dealing with multiple line segments, the approach reflected in Program 2.2 suffers from poor storage utilization because end points shared by several line segments are duplicated in each. Furthermore, that two line segments share an end point can only be detected by comparing the coordinate values, a slow and numerically risky operation. Another way


```
typedef struct
{
        point       p1;
        point       p2;
} line;

ex2()
{
        line        *l1;

        l1 = newline(0.0, 0.0, 0.0, 1.0, 0.0, 0.0);
}
```

Program 2.2 Type *line*.

of structuring the information that avoids this replaces multiple copies of a point by references to a single copy. In C, a natural way to implement this is to use separate records for points and line segments, and link them together with pointers. An implementation of types *point* and *line* that follows this approach is given in Program 2.3.

2.1.4 Design Issues

The difference between the two representations may seem to be small. Nevertheless, it is important and illustrates the nature of design decisions that must be done during the design of geometric systems. Some problems and issues are discussed below.

Description of Figures

Both data representations allow the implementation of similar procedures for the description of simple line figures, such as boxes, stroke-precision characters, and symbols. However, the "flat" design of Program 2.2 makes the implementation of certain kinds of operations inconvenient.

Consider, for instance, an operation where a box (four-sided rectilinear object whose sides are parallel to coordinate axes) is created by indicating its two opposite corners. In the first approach, the natural implementation reads two locations of a pointing device, and goes on to create the box. In the second approach it is natural to let the user indicate two existing points, and create the box using their stored coordinates. Obviously, it is much easier to create boxes that "match" in the second alternative.

```
typedef struct
{
        float       vx, vy, vz, vw;
} point;
typedef struct
{
        point       *p1;
        point       *p2;
} line;

ex3()
{
        point       *pnt1, *pnt2;
        line        *l1;

        pnt1 = newpoint(0.0, 0.0, 0.0);
        pnt2 = newpoint(1.0, 1.0, 1.0);
        l1 = newline(pnt1, pnt2);
}
```

Program 2.3 Another definition of *point* and *line*.

Manipulation of Figures

The differences of the two representations are accentuated as for the manipulation of figures. Consider a simple figure, such as the cottage in Figure 2.1.

If we modify the endpoints of the line segments so as to "stretch" the cottage, the first representation forces us to repeat the same modification for all line segments that meet at a point. In the second approach, we only need to modify the coordinates once, and all line segments would "follow" the change. The approach of storing the points separately also enhances the viewing of models, because each point needs to be projected only once.

As the second approach provides independent modeling primitives for points and lines, it makes various more advanced construction techniques for figures possible. We shall discuss these additional construction techniques further in Section 2.3.

Integrity

Small numerical errors can cause problems in the first approach. For instance, if line segments that seem to meet a point do not quite do so, an

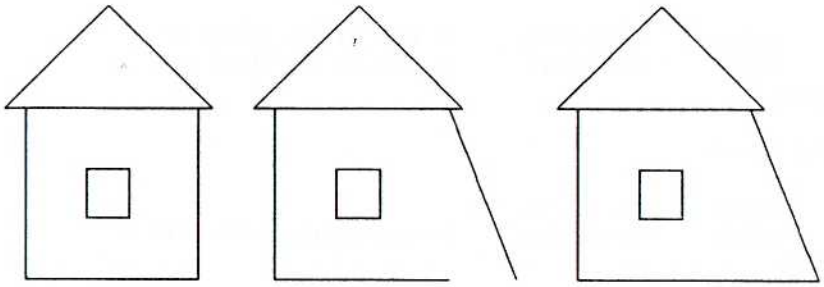

Figure 2.1 A house.

operation that attempts to move that point could erroneously leave a line segment behind.

On the other hand, the introduction of pointers has the drawback that we have also introduced the possibility of having "dangling" pointers that refer to incorrect information. For instance, suppose that in addition to procedures **newline** and **newpoint**, the corresponding *deletion* procedures **delline** and **delpoint** are needed for the interactive manipulation of figures. Obviously, **delpoint** should be applicable only if the point to be deleted does not appear in any line segments. In the data structure of Program 2.3, testing this condition is expensive because of the necessity to scan all line segments.

A straightforward way to improve the solution is to include a new attribute *usagecount* into *point*. Initially zero, its value is incremented each time **newline** creates a line segment emanating from the point. Similarly, **delpoint** can decrement *usagecount* for the end points of the line segment it deletes. Clearly, the procedure **delpoint** can work correctly by examining **usagecount**: if it equals zero, the point does not appear in any line segments, and can safely be deleted.

Another mechanism for handling the problem is based on including an access path from each point to the all line segments adjacent to it. One way to implement this is to associate with each point a pointer to a linked list of adjacent line segments. As each line segment will appear in exactly two such lists, they can be realized by including two new pointers to each line segment node. In this approach it becomes possible to implement a more powerful point deletion operation that removes a point and all its adjacent lines. We leave the details of this and the specification of the resulting modeling procedures as an exercise to the reader.

```
typedef struct
{
        point       *allpoints;      /* list of all points */
        line        *alllines;       /* list of all lines */
} object;

typedef struct
{
        float       vx, vy, vz, vw;
        point       *nextpoint;      /* next point of the list */
} point;

typedef struct
{
        point       *p1;
        point       *p2;
        line        *nextline;       /* next line of the list */
} line;
```

Program 2.4 Type *object*.

We choose the second representation for the purposes of this chapter; see [15] for additional discussion of the pros and cons of the two approaches.

2.2 GROUPING

The concepts of Program 2.3 give us the basic tools for creating graphical models of objects. In order to employ our models, we still need the capability of collecting logically interrelated points and line segments together into what might be called *groups*. This would allow our algorithms to process groups as a whole.

2.2.1 Objects

A new modeling primitive, *object* is needed to accomplish the grouping of related line segments and points. Objects consist of a set **allpoints** of points, and a set **alllines** of line segments. One way to implement this is to gather points and line segments belonging to the same set into linked lists, and let **allpoints** and **alllines** point to their heads. Definitions of *point* and *line* must then be modified to include a pointer to the next element in the list. These changes are implemented in Program 2.4.

Modeling procedures must be modified accordingly. One possibility is to include a pointer to the parent object as the first argument of procedures **newpoint** and **newline**, and let them add the new *point* or *line* to the correct list. Program 2.5 gives an example of the modeling procedures after these modifications.

2.2.2 Graphical Symbols

The grouping facility presented above is the simplest possible. A more interesting grouping mechanism allows the definition of *graphical symbols* that can be constructed through an *instantiation procedure* that assigns position and orientation information for a new *instance* of an object. In this extension, objects may consist of symbols, in addition to points and line segments.

A natural solution to this assigns a 4-by-4 transformation matrix that describes the position and orientation of each object instance relative to its parent object. (See Appendix A for information on 4-by-4 transformations.) Program 2.6 illustrates some possibilities offered by such a mechanism.

In the program, procedure **instanceof** creates a new instance of the object given as the first argument, transformed with the matrix given as the second argument. Such an instance can be added to an object with procedure **newinstance** analogously to procedures **newpoint** and **newline** of Program 2.4. Observe how the procedure can create a staircase pyramid from three instances of a single box.

The symbol data structure forms a directed acyclic graph which must be traversed when an object including symbols is "evaluated." In this process, the standard technique is to maintain a transformation stack during the traversal. Initially, an identity transformation is pushed onto the stack. When advancing to a child symbol, the current top of the stack is multiplied by the transformation of the child, and pushed onto the stack. This gives the correct transformation for positioning the child properly with respect to its parent. When the processing of a child is finished (perhaps after recursive evaluations of its children), the top of the transformation stack can be removed.

Symbol schemes are useful when pictures are formed from many instances of a small number of basic objects; consider, for instance, circuit diagrams. Correspondingly, techniques for symbol manipulation belong to the core of computer graphics methodology. We shall not delve deeper into graphical symbols; for a good exposition on the subject, the reader is referred to [39].

```
object  *newbox(dx, dy, dz)
float   dx, dy, dz;
{
        object      *box;
        point       *p1, *p2, *p3, *p4, *p5, *p6, *p7, *p8;

        /* make box an empty object */
        box = newobject();

        /* create points is the box */
        p1 = newpoint(box, 0.0, 0.0, 0.0);
        p2 = newpoint(box, dx, 0.0, 0.0);
        p3 = newpoint(box, dx, dy, 0.0);
        p4 = newpoint(box, 0.0, dy, 0.0);
        p5 = newpoint(box, 0.0, 0.0, dz);
        p6 = newpoint(box, dx, 0.0, dz);
        p7 = newpoint(box, dx, dy, dz);
        p8 = newpoint(box, 0.0, dy, dz);

        /* create lines of the box */
        newline(box, p1, p2);
        newline(box, p2, p3);
        newline(box, p3, p4);
        newline(box, p4, p1);
        newline(box, p5, p6);
        newline(box, p6, p7);
        newline(box, p7, p8);
        newline(box, p8, p5);
        newline(box, p1, p5);
        newline(box, p2, p6);
        newline(box, p3, p7);
        newline(box, p4, p8);

        return(box);
}
```

Program 2.5 Definition of a box.


```
typedef float matrix[4][4];

object  *pyramid()
{
        object      *b, *pyr1, *pyr2, *pyr3;
        instance    *s1, *s2, *s3, *s4;
        matrix      m1, m2;

        b = newbox(1.0, 1.0, 0.25);

        ident(m1);
        trans(m1, -0.5, -0.5, 0.0);
        ident(m2);
        scale(m2, 1.2, 1.2, 1.0);
        trans(m2, 0.0, 0.0, -0.25);

        pyr1 = newobject();
        s1 = instanceof(b, m1);
        newinstance(pyr1, s1);

        pyr2 = newobject();
        s2 = instanceof(pyr1, m2);
        newinstance(pyr2, s1);
        newinstance(pyr2, s2);

        pyr3 = newobject();
        s3 = instanceof(pyr2, m2);
        newinstance(pyr3, s1);
        newinstance(pyr3, s3);
}
```

Program 2.6 Pyramid.

2.3 EXTENSIONS

The very basic functionality developed so far is adequate only for illustrating the problems of, and techniques for graphical models. A practical system should include various extensions both for increased functionality and ease of use. Some of these extensions are discussed below.

2.3.1 Construction Techniques

For the time being, we can construct graphical models from two modeling primitives, *point* and *line* that can be grouped to form *objects*. Our modeling system is capable of

1. creating new *points* at desired locations,
2. creating *lines* between two existing *points*, and
3. scanning through *lines* and *points* of an *object*.

Clearly, by just these constructs we can model all possible line figures, so as for expressive power, we are done! However, for practical purposes many other construction methods for points and line segments would be useful as well. These additional constructions may include the following:

1. In plane, construct a line segment parallel to a given line, of given length, and starting from a given point. Length is measured along the direction of the argument line.
2. Construct a line segment perpendicular to a given line, of given length, and starting from a given point. The direction of the line segment is chosen to be 90 degrees counterclockwise from the direction of the argument line.
3. Construct a point as the intersection of two lines.
4. Construct a point as the mirrored image of a point with respect to a line.

Note that some of these operations may create many points and line segments in one shot.

Commercial drafting systems include all these and many other construction methods as well, including functionality for creating circles and arcs in terms of existing points, lines, and arcs. See [105] for a compact account of various construction methods and their implementations.

2.3.2 Systems With Constraints

Suppose the additional construction operations described in Section 2.3.1 above are implemented on top of the data structure of Program 2.4 (perhaps extended as suggested in Section 2.1.4). Let us consider the implementation of a new procedure `modifypoint(p, newx, newy, newz)` that assigns new coordinates to point.

Suppose further that a line segment l_1 starting from point p and of given length is created to be parallel to a line l_0. What happens if we update the coordinates of p? After the operation, l_1 (and all other line segments emanating from p) has been modified, and in all likelihood l_1 is not anymore parallel to l_0.

This effect can be perfectly acceptable in some applications. Sometimes, however, we would like to see the line segment l_1 move with p while remaining parallel to l_0. Hence, the other end point of l_1 must move as well, and any line segments potentially emanating from it move according to how they were constructed. This facility is useful, say, when dealing with *rigid* objects whose shape is invariant under movement.

Representations for Constraints

To implement such a scheme we must be able to associate constraints with each line segment and point. For simplicity, let us discuss how we could store the constraints in the case of just two construction methods of line segments, namely

1. construct a line segment between two points
2. construct a line segment parallel to another line, starting at a point, and of given length.

Observe that the latter construction creates both a line segment and a point. It can be implemented as first constructing the other end point of the line segment as determined by the arguments, and then constructing the resulting line segment between the two end points.

Constrained graphical models generally break the representation of each entity into two parts. Data that are independent of the construction method by which the entity was created are referred to as the *canonical* representation of the entity, whereas the remaining data forms the *reference* representation of the entity.

Let us use the representation of Program 2.4 as the canonical representation of our line segments and points. To include the reference information into line segments and points, we add a new attribute *refs* for storing a pointer to a node that records the construction method used for the

```
typedef struct
{
        int         code;           /* = 0 */
        float       x, y, z;
} point_from_coordinates;

typedef struct
{
        int         code;           /* = 1 */
        line        *l;
        point       *p;
        float       *l;
} point_at_distance_from_point_along_line;
```

Program 2.7 Points with constraints.

line segment or the point. These nodes include a code that identifies the particular construction method they represent, and a list of arguments of the construction procedure. The arguments may be pointers to geometric entities or numerical values.

As our line segments are always bounded by two points, it turns out that only points need to be constrained. In our example, we need the two constraint nodes depicted in Program 2.7 for the two different kinds of points: (1) those that simply are created from a triple of coordinates (like in `newpoint`), and (2) those that are created in the construction above as the other end point of the parallel line segment.

Regeneration

Let us now return to the implementation of `modifypoint` with the constrained data structures of Program 2.7. Obviously, the operation is meaningful only if the point is of type (1) above (why?). After checking this, the algorithm should *regenerate* all line segments and points whose canonical representation depends on the modified point either directly or through a sequence of any constructions.

Observe that the construction information defines a partial order of line segments and points[1]. Let us write $a < b$ to denote that a is defined in terms of b. To perform the regeneration operation correctly, we must process all objects $e < p$, the modified point. Moreover, we must assure

[1]This is so because the constrained model was created by a linear sequence of modeling operations in the first place.

that if $a < b$, b is regenerated before a.

One way to accomplish this is to link all points and line segments into yet another linked list in the order they were created. The regeneration algorithm can then consider points and line segments in the order they appear in the list. However, this solution has the drawback that lots of unnecessary work is performed if only a few entities must be updated. At the cost of additional stored information, a more efficient approach is to perform a search in the graph defined by constrain relationships while taking the relation $<$ into account.

Unbounded Graphical Entities

Some graphical modelers make a distinction of *bounded* graphical entities (such as our line segments) and *unbounded* graphical entities. In these systems lines are the truly unbounded lines of mathematics, whereas a separate entity is used for representing bounded line segments.

While increasing the complexity of the modeler, this separation allows the inclusion of additional construction methods and simplifies the specification of many constructions. For instance, a parallel line could be constructed just by giving a distance from the argument line, whereas a starting point and a length would be additionally required for the creation of a bounded line segment.

Observe that if unbounded graphical entities are used, both lines and points would be constrained. For instance, a parallel line constructed as above would depend on the argument line. Points created on the line would depend on it, and line segments adjacent to those points on them.

2.3.3 Parametric Design

Constrained graphical data structures bring us to data structures useful for a *drafting system*. A drafting system allows the user to interactively construct figures from simple geometric entities. A full-scale system would include all facilities discussed in this chapter, including grouping (perhaps in several ways), symbols, many construction methods, and constrained data structures. It would probably include additional modeling primitives, such as arcs and character strings. It would also include functional components not discussed here at all, such as a graphical user interface and mechanisms for storing models into secondary storage.

One interesting characteristic of some drafting systems is the capability of supporting *parametric design*. It can be understood as a generalization of constrained data structures. Consider again the example of Program 2.7. Going a step further, let us replace the number *length* by a pointer to a

new entity *number*. This gives us the capability of storing *all* dimensions of an object as *numbers*, and referring to them in other constructions.

In fact, we can include construction methods for numbers in our modeling system. These constructions are likely to include simple mathematical expressions and geometric measurements such as distance. Like other entities, each number has a canonical format that simply gives its value, and a constrained format that represents the expression or measurement from which the number was generated. Extending the regeneration algorithm to numbers gives the capability of modifying the (few) numbers that form the roots of the whole construction, and hence generating new versions of the whole design easily.

PROBLEMS

2.1. Write a C definition of points and lines as suggested in Section 2.1.4. That is, associate with each point the collection of lines it is adjacent to.

2.2. Write modified versions of procedures `newline` and `newpoint` that use the modified data structures of Problem 1.

2.3. Specify and write procedures `delline` and `delpoint` based on the data structures of Problem 1.

2.4. Based on the data structures of Program 2.5, write procedures `newobject`, `instanceof`, and `newinstance` discussed in Section 2.2.

2.5. Based on the data structures of Program 2.7, specify and write the procedures needed for drawings with constraints.

BIBLIOGRAPHIC NOTES

For this book, graphical models are only of secondary interest. A reader interested on them should therefore consult additional literature.

Graphical models are the major ones currently in use in Computer-Aided Design and Drafting systems. Consequently, books on CADD [15,11,43] contain material on graphical models much beyond this chapter.

The texts of Faux and Pratt [38] and Mortenson [85] contain (in addition to a lot of other material) good overviews of curve and surface presentations. The book by Woodwark [131] gives a compact account on not only two-dimensional techniques, but also on many aspects on three-dimensional models as well. The books of Chasen [24], Rogers [103,102], and Bowyer and Woodwark [16] contain readily useful techniques for various geometric constructions (primarirly) for simple geometric objects. Commercially available CADD systems are the subject of [32].

Parametric design techniques are in use not only in drafting systems but also in the user interfaces of solid modelers. For instance, the user interface of MEDUSA [87] has a parametric design facility that includes also circular arcs and is capable of storing relationships involving tangency constraints. As a more advanced example, Lin *et al.* [71] give an algorithm that can check whether a set of such constraints fully determines a two-dimensional figure consisting of line segments and arcs. The algorithm can also flag overconstrained figures.